

// Declaración de un ArrayList de "String". Puede ser de cualquier otro Elemento u Objeto (float, Boolean, Object, ...)

```
ArrayList<String> nombreArrayList = new ArrayList<String>();
```

// Añade el elemento al ArrayList

```
nombreArrayList.add("Elemento");
```

// Añade el elemento al ArrayList en la posición 'n'

```
nombreArrayList.add(n, "Elemento 2");
```

// Devuelve el numero de elementos del ArrayList

```
nombreArrayList.size();
```

// Devuelve el elemento que esta en la posición '2' del ArrayList

```
nombreArrayList.get(2);
```

// Comprueba se existe del elemento ('Elemento') que se le pasa como parametro

```
nombreArrayList.contains("Elemento");
```

// Devuelve la posición de la primera ocurrencia ('Elemento') en el ArrayList

```
nombreArrayList.indexOf("Elemento");
```

// Devuelve la posición de la última ocurrencia ('Elemento') en el ArrayList

```
nombreArrayList.lastIndexOf("Elemento");
```

// Borra el elemento de la posición '5' del ArrayList

```
nombreArrayList.remove(5);
```

// Borra la primera ocurrencia del 'Elemento' que se le pasa como parametro.

```
nombreArrayList.remove("Elemento");
```

//Borra todos los elementos de ArrayList

```
nombreArrayList.clear();
```

// Devuelve True si el ArrayList esta vacio. Sino Devuelve False

```
nombreArrayList.isEmpty();
```

// Copiar un ArrayList

```
ArrayList arrayListCopia = (ArrayList) nombreArrayList.clone();
```

// Pasa el ArrayList a un Array

```
Object[] array = nombreArrayList.toArray();
```

Matrices en Java

Una matriz es una colección de tipos de datos similares. También es llamada arreglo multidimensional.

Su sintaxis es la siguiente:

```
Int[][] matriz_ejemplo = new int[3][4];
```

	Column 1	Column 2	Column 3	Column 4
Row 1	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 2	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 3	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Recordemos que la indexación en Java siempre empieza en 0.

```
class MultidimensionalArray {
    public static void main(String[] args) {

        // create a 2d array
        String [][] matriz_ejemplo = {
            {"A1", "B2", "C3"},
            {"D4", "E5", "F6"},
            {"G7", "H8", "I9"},
            {"J10", "K11", "L12"}
        };

        // calculando la longitud de cada fila, o elementos que contiene cada fila
        System.out.println("Elementos de la fila 1: " + matriz_ejemplo[0].length);
        System.out.println("Elementos de la fila 2: " + matriz_ejemplo[1].length);
        System.out.println("Elementos de la fila 3: " + matriz_ejemplo[2].length);

        // Imprimiendo el número de columnas (que es igual al número de filas)
        System.out.println("Número de columnas: " + matriz_ejemplo.length);

        // Imprimiendo elementos
        for (int i = 0; i < matriz_ejemplo.length; ++i) {
            for (int j = 0; j < matriz_ejemplo[i].length; ++j) {
                System.out.println(matriz_ejemplo[i][j]);
            }
        }
    }
}
```

Las **pilas** y las **colas** son estructuras de datos fundamentales en Java que permiten el almacenamiento y manejo eficiente de información. Estas estructuras siguen principios lógicos diferentes, lo que les confiere aplicaciones y propiedades únicas.

Pilas en Java

import java.util.Stack; //Se utiliza esta clase predefinida, para crear una pila

```
public class EjemploPila {
    public static void main(String[] args) {
        // Crear una pila
        Stack<Integer> pila = new Stack<>();

        // Insertar 7 elementos en la pila
        pila.push(10); //Método definido en la clase Stack, se utiliza para añadir elementos
        pila.push(20);
        pila.push(30);
        pila.push(40);
        pila.push(50);
        pila.push(60);
        pila.push(70);

        // Mostrar la pila
        System.out.println("Elementos en la pila: " + pila);

        // Sacar elementos de la pila
        System.out.println("Elemento eliminado: " + pila.pop());
        System.out.println("Elemento eliminado: " + pila.pop()); //pop para ubicarse en la cima de
        la pila

        // Mostrar la pila después de eliminar elementos
        System.out.println("Pila después de eliminar dos elementos: " + pila);

        // Ver el elemento en la parte superior de la pila sin eliminarlo
        System.out.println("Elemento en la cima de la pila: " + pila.peek()); //visualización del
        elementos de la parte superior

        // Verificar si la pila está vacía
        System.out.println("¿Está vacía la pila? " + pila.isEmpty()); //método para verificar si hay
        más elementos de la pila
    }
}
```

Colas en Java

```
import java.util.LinkedList; //Clase necesaria que usa nodos enlazados
import java.util.Queue; //Clase necesaria para especificar la estructura de colas

public class EjemploCola {
    public static void main(String[] args) {
        // Crear una cola
        Queue<Integer> cola = new LinkedList<>();

        // Insertar 7 elementos en la cola
        cola.offer(10); //método para añadir elementos
        cola.offer(20);
        cola.offer(30);
        cola.offer(40);
        cola.offer(50);
        cola.offer(60);
        cola.offer(70);

        // Mostrar la cola
        System.out.println("Elementos en la cola: " + cola);

        // Eliminar elementos de la cola
        System.out.println("Elemento removido: " + cola.poll()); //elimina el PRIMERO que entró
        System.out.println("Elemento removido: " + cola.poll());

        // Mostrar la cola después de eliminar elementos
        System.out.println("Cola después de eliminar dos elementos: " + cola);

        // Ver el elemento en el frente de la cola sin eliminarlo
        System.out.println("Elemento en el frente de la cola: " + cola.peek()); //visualización del
        elementos de la parte superior

        // Verificar si la cola está vacía
        System.out.println("¿Está vacía la cola? " + cola.isEmpty()); //método para verificar si hay
        más elementos de la pila
    }
}
```